Lab 8: Integrating External REST APIs (Live Book Metadata & Currency Conversion)

Lab Overview and Architecture

In **Lab 7**, you exposed your local SQLite database to outside clients by building BooksApiController (acting as an **API Producer**). In **Lab 8**, your ASP.NET Core MVC application reverses roles and acts as an **API Consumer (HTTP Client)**.

When a user clicks 🌐 **Live Details** on any book in your catalog, BooksController will:

1. Query the local **SQLite Database** (AppDbContext) for the book's core record (Title, Author, PublishedYear, and Price in USD).

2. Call the **OpenLibrary REST API** (https://openlibrary.org/search.json) to fetch live global publishing metadata (official book cover image, total worldwide editions, and first publication year).

3. Call the **Open Exchange Rates API** (https://open.er-api.com/v6/latest/USD) to fetch the live **USD to INR (₹)** exchange rate and calculate the book's real-time price in Indian Rupees.

4. Combine all three data sources into a single BookDetailsViewModel and render them in Views/Books/Details.cshtml—while using **Graceful Degradation** so the page still loads local SQLite data even if the internet connection drops.

Data Source	Endpoint / Origin	Data Retrieved	Failure Behavior
Local SQLite DB	AppDbContext.Books	Id, Title, Author, PublishedYear, Price ($)	Returns 404 Not Found if ID does not exist
OpenLibrary API	https://openlibrary.org/search.json	Book cover image, global edition count, first publish year	Displays local book data + warning banner
Exchange Rate API	https://open.er-api.com/v6/latest/USD	Live USD →INR conversion rate	Displays USD price only + warning banner

Why These Two Public APIs? Both **OpenLibrary** and **Open Exchange Rates (open.er-api.com)** are free, open-access REST APIs that **do not require API keys or account registration**, allowing every student to run and test the code immediately.

Step 1: Create the DTOs and ViewModel (Models/BookDetailsViewModel.cs)

External APIs return large, nested JSON payloads using snake_case naming conventions (such as first_publish_year and edition_count). Rather than polluting your SQLite database entity (Models/Book.cs), we create lightweight **Data Transfer Objects**

(DTOs) mapped with [JsonPropertyName] to deserialize only the fields we need, alongside a **ViewModel** that bundles everything for the Razor View.

Create a new file named BookDetailsViewModel.cs inside your Models folder and paste the following code:

C#

```
None
using System.Text.Json.Serialization;

namespace MyMvcApp.Models
{
    // 1. DTO for the OpenLibrary Search API Root JSON
Response
    // Endpoint:
https://openlibrary.org/search.json?title=...&limit=1
    public class OpenLibrarySearchResponse
    {
        [JsonPropertyName("numFound")]
        public int TotalMatches { get; set; }

        [JsonPropertyName("docs")]
        public List<OpenLibraryBookDto> Docs { get; set; } =
new();
    }

    // 2. DTO for a single Book document inside OpenLibrary's
"docs" array
    public class OpenLibraryBookDto
    {
        [JsonPropertyName("first_publish_year")]
        public int? FirstPublishYear { get; set; }

        [JsonPropertyName("edition_count")]
        public int EditionCount { get; set; }
```

```
        [JsonPropertyName("cover_i")]
        public int? CoverImageId { get; set; }
    }

    // 3. DTO for the Currency Exchange API JSON Response
    // Endpoint: https://open.er-api.com/v6/latest/USD
    public class ExchangeRateApiResponse
    {
        [JsonPropertyName("result")]
        public string Result { get; set; } = string.Empty;

        [JsonPropertyName("base_code")]
        public string BaseCode { get; set; } = string.Empty;

        [JsonPropertyName("rates")]
        public Dictionary<string, decimal> Rates { get; set; }
= new();
    }

    // 4. Master ViewModel combining SQLite Book + OpenLibrary
Metadata + INR Currency Conversion
    public class BookDetailsViewModel
    {
        // Local SQLite Entity
        public Book LocalBook { get; set; } = default!;

        // External API 1: OpenLibrary Metadata
        public OpenLibraryBookDto? ExternalMetadata { get;
set; }

        // Helper property to build the cover image URL when
CoverImageId is available
```

```
        public string? CoverImageUrl =>
ExternalMetadata?.CoverImageId != null
            ?
$"https://covers.openlibrary.org/b/id/{ExternalMetadata.CoverI
mageId}-M.jpg"
            : null;

        // External API 2: Live USD to INR Currency Conversion
        public decimal? UsdToInrRate { get; set; }
        public decimal? PriceInInr => UsdToInrRate.HasValue
            ? Math.Round(LocalBook.Price * UsdToInrRate.Value,
2)
            : null;

        // Graceful Degradation warning message if either
external API is unreachable
        public List<string> ApiWarnings { get; set; } = new();
    }
}
```

How the Mapping Works:

- **[JsonPropertyName("...")]**: Connects external JSON keys (like "edition_count") directly to clean PascalCase C# properties (EditionCount).

- **Dictionary<string, decimal> Rates**: The currency API returns a JSON object containing over 160 currency codes ({ "USD": 1, "EUR": 0.92, "INR": 84.50, ... }). Deserializing it into a C# Dictionary<string, decimal> lets us look up Rates["INR"] in $O(1)$ time.

- **Computed Properties (CoverImageUrl and PriceInInr)**: Automatically calculate the image URL and Rupee price whenever the underlying API data is present, keeping our Razor View clean.

Step 2: Register Named HttpClient Services in Program.cs

Instantiating new HttpClient() manually inside a controller can exhaust operating system network sockets under load. Instead, we register **Named HTTP Clients** in Program.cs using builder.Services.AddHttpClient(...). This allows ASP.NET Core to pool network connections, set a strict 5-second timeout, and attach a required User-Agent header.

Open Program.cs and add the two AddHttpClient registrations right after builder.Services.AddControllersWithViews();:

C#

```
None
builder.Services.AddControllersWithViews();

// --- [NEW FOR LAB 8]: Register Named HttpClients for
External APIs ---

// 1. HTTP Client for OpenLibrary Book Metadata API
builder.Services.AddHttpClient("OpenLibraryClient", client =>
{
    client.BaseAddress = new Uri("https://openlibrary.org/");
    client.Timeout = TimeSpan.FromSeconds(5); // Fail fast if
external API is slow
    client.DefaultRequestHeaders.Add("User-Agent",
"MyMvcApp-IUST-FullStackLab/1.0");
});

// 2. HTTP Client for Live USD-to-INR Currency Exchange API
builder.Services.AddHttpClient("CurrencyClient", client =>
{
    client.BaseAddress = new Uri("https://open.er-api.com/");
    client.Timeout = TimeSpan.FromSeconds(5);
    client.DefaultRequestHeaders.Add("User-Agent",
"MyMvcApp-IUST-FullStackLab/1.0");
});
```

```
//
------------------------------------------------------------
-------
```

Step 3: Inject IHttpClientFactory & Add the Details Action (Controllers/BooksController.cs)

Next, update Controllers/BooksController.cs to:

1. Inject IHttpClientFactory into the constructor alongside AppDbContext.
2. Add an [AllowAnonymous] action named Details(int id) mapped to [HttpGet("{id:int}/Details")].
3. Wrap each external API call in its own try-catch block so that if one external service fails (or the lab loses internet connectivity), the other service and your local SQLite data still render normally.

Open Controllers/BooksController.cs and update the top of the class and add the Details method as shown below:

C#

```
None
using System.Net.Http.Json; // Required for GetFromJsonAsync
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MyMvcApp.Data;
using MyMvcApp.Models;

namespace MyMvcApp.Controllers
{
    [Authorize]
    [Route("Library/Books")]
    public class BooksController : Controller
```

```
    {
        private readonly AppDbContext _context;
        private readonly IHttpClientFactory
_httpClientFactory; // [NEW FOR LAB 8]

        // Update constructor to inject both AppDbContext and
IHttpClientFactory
        public BooksController(AppDbContext context,
IHttpClientFactory httpClientFactory)
        {
            _context = context;
            _httpClientFactory = httpClientFactory;
        }

        // ... keep your existing Index, Search, Create, Edit,
and Delete actions ...

        // --- [NEW FOR LAB 8]: Enriched Book Details with
External APIs ---
        // GET: /Library/Books/5/Details
        [AllowAnonymous]
        [HttpGet("{id:int}/Details")]
        public async Task<IActionResult> Details(int id)
        {
            // 1. Fetch core book record from local SQLite
database
            var book = await _context.Books.FindAsync(id);
            if (book == null)
            {
                return NotFound();
            }

            var viewModel = new BookDetailsViewModel
```

```
            {
                LocalBook = book
            };

            // 2. Call External API #1: OpenLibrary API (Book
Cover & Global Editions)
            try
            {
                var bookClient =
_httpClientFactory.CreateClient("OpenLibraryClient");

                // URL-encode the title so spaces and special
characters (e.g., "C# & .NET") don't break the URL
                string encodedTitle =
Uri.EscapeDataString(book.Title);
                var openLibResponse = await
bookClient.GetFromJsonAsync<OpenLibrarySearchResponse>(

$"search.json?title={encodedTitle}&limit=1");

                if (openLibResponse != null &&
openLibResponse.Docs.Any())
                {
                    viewModel.ExternalMetadata =
openLibResponse.Docs.First();
                }
            }
            catch (Exception)
            {
                viewModel.ApiWarnings.Add("OpenLibrary API is
currently unreachable. Live cover art and global edition
counts are hidden.");
            }
```

```
            // 3. Call External API #2: Currency Exchange API
(Live USD -> INR Rate)
            try
            {
                var currencyClient =
_httpClientFactory.CreateClient("CurrencyClient");
                var rateResponse = await
currencyClient.GetFromJsonAsync<ExchangeRateApiResponse>(
                    "v6/latest/USD");

                if (rateResponse != null &&
rateResponse.Rates.TryGetValue("INR", out decimal inrRate))
                {
                    viewModel.UsdToInrRate = inrRate;
                }
            }
            catch (Exception)
            {
                viewModel.ApiWarnings.Add("Currency Exchange
API is currently unreachable. Showing local USD price only.");
            }

            return View(viewModel);
        }
    }
}
```

Key Engineering Details to Note:

- **Uri.EscapeDataString(book.Title)**: If a book title contains spaces or symbols (like "Clean Code" or "ASP.NET & C#"), raw string interpolation would corrupt the HTTP query string. Uri.EscapeDataStringconverts it safely to "Clean%20Code".

- **GetFromJsonAsync<T>(...)**: Sends the HTTP GET request, checks that the response status code is 200 OK, reads the JSON stream asynchronously, and

deserializes it into our C# DTO in one line.

- **Independent try-catch Blocks**: Separating the OpenLibrary call from the Currency call ensures that if OpenLibrary times out, the student still sees the live USD-to-INR conversion, and vice versa.

Step 4: Create the Enriched Details View (Views/Books/Details.cshtml)

Create a new file named Details.cshtml inside your Views/Books folder. This view displays the local SQLite data alongside the live INR price badge and OpenLibrary metadata card:

HTML

```
None
@model MyMvcApp.Models.BookDetailsViewModel
@{
    ViewData["Title"] = $"{Model.LocalBook.Title} - Live
Details";
}

<div class="d-flex justify-content-between align-items-center
mb-4 mt-2">
    <div>
        <h2 class="fw-bold text-dark mb-0">📖 Book Catalog &
Global Insights</h2>
        <p class="text-muted mb-0">Combining local SQLite
storage with live REST APIs</p>
    </div>
    <a asp-action="Index" class="btn btn-outline-secondary
rounded-pill px-4 fw-semibold">
        ← Back to Catalog
    </a>
</div>
```

```
<!-- Graceful Degradation Warning Banner (Only shown if an
external API fails/times out) -->
@if (Model.ApiWarnings.Any())
{
    <div class="alert alert-warning shadow-sm rounded-3 mb-4">
        <h6 class="fw-bold mb-1">⚠️ External API Notice
(Graceful Degradation Active)</h6>
        <ul class="mb-0 ps-3">
            @foreach (var warning in Model.ApiWarnings)
            {
                <li>@warning</li>
            }
        </ul>
    </div>
}

<div class="card shadow-sm border-0 rounded-3 overflow-hidden
bg-white">
    <div class="card-header bg-dark text-white py-3 px-4
d-flex justify-content-between align-items-center">
        <h4 class="mb-0 fw-bold">@Model.LocalBook.Title</h4>
        <span class="badge bg-secondary rounded-pill px-3
py-2">SQLite ID: #@Model.LocalBook.Id</span>
    </div>

    <div class="card-body p-4">
        <div class="row g-4 align-items-center">

            <!-- LEFT COLUMN: Live Book Cover from OpenLibrary
API -->
            <div class="col-md-4 text-center">
                @if (Model.CoverImageUrl != null)
                {
```

```
                    <img src="@Model.CoverImageUrl"
                         alt="Cover of @Model.LocalBook.Title"
                         class="img-fluid rounded-3 shadow"
                         style="max-height: 300px; object-fit:
cover;" />
                    <div class="mt-2">
                        <span class="badge bg-light
text-secondary border">Fetched live from OpenLibrary</span>
                    </div>
                }
                else
                {
                    <div class="bg-light border rounded-3 p-5
text-muted d-flex flex-column justify-content-center
align-items-center" style="min-height: 260px;">
                        <span class="fs-1 mb-2">📚</span>
                        <span class="fw-semibold">No Live
Cover Found</span>
                        <small class="text-secondary
mt-1">OpenLibrary has no cover image for this title</small>
                    </div>
                }
            </div>

            <!-- RIGHT COLUMN: Local SQLite Data + Currency
API + OpenLibrary Metadata -->
            <div class="col-md-8">

                <!-- Section 1: Local SQLite Record -->
                <h5 class="text-primary fw-bold mb-3">🗄️
Local Database Record (SQLite)</h5>
                <div class="row mb-3">
                    <div class="col-sm-6 mb-2">
```

```
                        <span class="text-muted d-block
small">AUTHOR</span>
                        <span class="fw-semibold fs-5
text-dark">@Model.LocalBook.Author</span>
                    </div>
                    <div class="col-sm-6 mb-2">
                        <span class="text-muted d-block
small">LOCAL CATALOG YEAR</span>
                        <span class="fw-semibold fs-5
text-dark">@Model.LocalBook.PublishedYear</span>
                    </div>
                </div>

                <hr class="my-3" />

                <!-- Section 2: Live Currency Conversion API
(USD -> INR) -->
                <h5 class="text-success fw-bold mb-3">💱 Live
Pricing & Currency Conversion API</h5>
                <div class="row g-3 mb-3">
                    <div class="col-sm-6">
                        <div class="p-3 bg-light rounded-3
border">
                            <span class="text-muted d-block
small">BASE PRICE (USD)</span>
                            <span class="fs-4 fw-bold
text-dark">$@Model.LocalBook.Price.ToString("0.00")</span>
                        </div>
                    </div>
                    <div class="col-sm-6">
                        <div class="p-3 bg-success
bg-opacity-10 rounded-3 border border-success">
```

```
                            <span class="text-success
fw-semibold d-block small">LIVE CONVERTED PRICE (INR)</span>
                            @if (Model.PriceInInr.HasValue &&
Model.UsdToInrRate.HasValue)
                            {
                                <span class="fs-4 fw-bold
text-success">₹@Model.PriceInInr.Value.ToString("N2")</span>
                                <small class="d-block
text-muted mt-1">
                                    Live Rate: 1 USD =
₹@Model.UsdToInrRate.Value.ToString("0.00")
                                </small>
                            }
                            else
                            {
                                <span class="text-muted
fst-italic">Conversion unavailable</span>
                            }
                        </div>
                    </div>
                </div>

                <hr class="my-3" />

                <!-- Section 3: Live OpenLibrary Global
Metadata -->
                <h5 class="text-info fw-bold mb-3">🌐 Global
Publishing Metadata (OpenLibrary API)</h5>
                @if (Model.ExternalMetadata != null)
                {
                    <div class="row">
                        <div class="col-sm-6 mb-2">
```

```
                            <span class="text-muted d-block
small">FIRST PUBLISHED WORLDWIDE</span>
                            <span class="fw-semibold
text-dark">

@(Model.ExternalMetadata.FirstPublishYear?.ToString() ?? "Not
Listed")
                            </span>
                        </div>
                        <div class="col-sm-6 mb-2">
                            <span class="text-muted d-block
small">TOTAL GLOBAL EDITIONS</span>
                            <span class="badge bg-info
text-dark rounded-pill px-3 py-2 fw-bold">

@Model.ExternalMetadata.EditionCount Worldwide Editions
                            </span>
                        </div>
                    </div>
                }
                else
                {
                    <p class="text-muted fst-italic mb-0">
                        No matching global publication record
found on OpenLibrary for "@Model.LocalBook.Title".
                    </p>
                }

            </div>
        </div>
    </div>
</div>
```

Step 5: Add the "🌐 Live Details" Button to the Catalog (Views/Books/Index.cshtml)

Open Views/Books/Index.cshtml and locate the <td class="py-3 text-end px-4"> Actions column inside the @foreach (var item in Model) loop. Add the 🌐 **Live Details** Anchor Tag Helper right above the @if (User.Identity.IsAuthenticated) block so that all visitors can inspect the live API details:

HTML

```
None
<td class="py-3 text-end px-4">
    <!-- [NEW FOR LAB 8]: Public button to view live
OpenLibrary & Currency API data -->
    <a asp-action="Details" asp-route-id="@item.Id" class="btn
btn-sm btn-outline-info rounded-pill px-3 fw-semibold me-1">
        🌐 Live Details
    </a>

    <!-- Existing Authenticated Edit Button -->
    @if (User.Identity!.IsAuthenticated)
    {
        <a asp-action="Edit" asp-route-id="@item.Id"
class="btn btn-sm btn-outline-primary rounded-pill px-3
fw-semibold">Edit</a>
    }

    <!-- Existing Admin Delete Button -->
    @if (User.IsInRole("Admin"))
    {
        <a asp-action="Delete" asp-route-id="@item.Id"
class="btn btn-sm btn-outline-danger rounded-pill px-3
fw-semibold ms-1"
           onclick="return confirm('Are you sure you want to
delete this book?');">Delete</a>
    }
```

```
</td>
```

Step 6: Verification and Classroom Testing

1. Start your application server in the terminal:

2. Bash

```
None
dotnet run
```

3.
4. Open your browser to http://localhost:5151/Library/Books.

5. If your catalog is empty, log in (admin@mylibrary.com / Admin@123!) and add a well-known book such as:

    - **Title:** Clean Code
    - **Author:** Robert C. Martin
    - **Published Year:** 2008
    - **Price:** 40.00

6. **Test 1 — Live Dual-API Integration (Online):**

    - Click the 🌐 **Live Details** button next to Clean Code.
    - Verify that the page displays:

        1. The official book cover image loaded dynamically from OpenLibrary.
        2. The base price ($40.00) alongside the live converted Indian Rupee price (e.g., ₹3,460.00) and the real-time 1 USD = ₹... exchange rate.

3. The global edition count and first publication year from OpenLibrary.

7. **Test 2 — Graceful Degradation (Offline Resilience Test):**

    - Turn off your computer's Wi-Fi (or disconnect the network) and refresh the /Library/Books/1/Detailspage.

    - Instead of crashing with an unhandled HttpRequestException, the page catches the network error, displays the yellow ⚠️ **External API Notice** banner at the top, and continues rendering the local SQLite book record (Title, Author, PublishedYear, and $40.00 USD price).